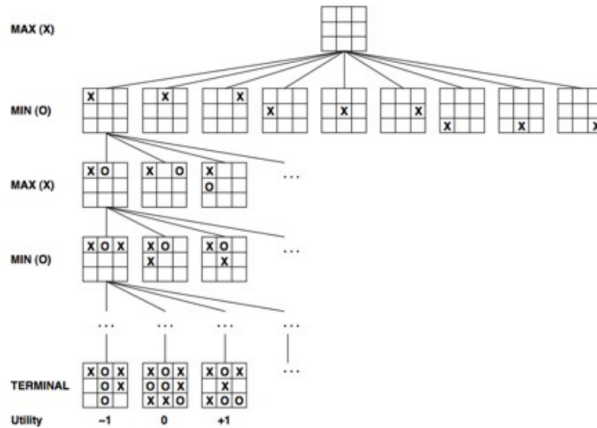


Lecture 5 -- Adversarial Search

- 对抗性搜索 (Adversarial Search)
 - 对抗性搜索问题 $\equiv$ 博弈 (games)
 - 它们出现在多智能体的竞争性环境中，存在一个我们无法控制、会针对我们做出计划的对手 (opponent)
 - 与搜索不同：博弈的最优解不是一连串固定动作，而是一条策略 (policy) 。
 - 例如：如果对手做 a，代理做 b；如果对手做 c，代理做 d；以此类推。
 - 如果用硬编码规则实现策略会既乏味又脆弱 (容易失效)
 - 博弈可以被建模为搜索问题，并使用启发式评估函数 (heuristic evaluation functions)
 - Zero-sum Games — 零和博弈
 - 对抗性 (Adversarial)：纯竞争。
 - 不同玩家对结果有不同的价值判断
 - 一名玩家要最大化某个单一数值，而另一名玩家试图最小化它。
 - 一个函数：一名玩家最大化它，另一名玩家则最小化它！
 - 玩家每走一步称为一个“ply” (半步)。
 - Embedded thinking — 嵌套 (内嵌) 思考
 - 嵌套思考或逆向推理 (Embedded thinking or backward reasoning)
 - 要点：
 - 一名玩家试图弄清楚应该做什么。如何决定？他考虑可能动作的后果。他还需要考虑对手会怎么想/怎么做…。对手也在考虑下一步该如何应对。双方都会想象对方对自己动作的回应。
 - 这就产生了嵌套的思考 (embedded thinking)。
 - Formalization — 形式化
 - 初始状态 (The initial state)
 - Player(s)：定义在状态 s 哪一方有权移动 (通常交替进行)。
 - Actions(s)：返回在状态 s 下的合法走法集合。
 - Transition function： $S \times A \rightarrow S$ ，定义一次走子导致的结果状态。
 - Terminal test：当游戏结束时为 True，否则为 False。游戏结束的那些状态称为终结态 (terminal states)。
 - Utility(s, p)：对在终结态 s 下的玩家 p 的效用函数或目标函数。
 - 例如在国际象棋中，结果是胜/负/和，可取值 +1、0、-1；在井字棋中可以用 +1、-1、0 等效用值。
 - minimax

- 两名玩家 Max 与 Min，玩家交替走子，Max 先手，Max 试图使结果最大化，Min 试图使结果最小化
- 计算每个节点的 minimax 值：即在对手也最优的情况下能取得的最佳收益

Minimax example



- 寻找 Max 的最优策略：
 - 对游戏树进行深度优先搜索 DFS
 - 最优的叶节点可能出现在树中的任意深度
 - Minimax 原理：假定双方从当前状态起直到终局都采用最优策略，计算处于该状态时的效用
 - 一旦发现终结节点，就将它们的 minimax 值向上回传（在树上向上传播）
- minimax 的规则

For a state s $\text{minimax}(s) =$

对于任意状态 s , $\text{minimax}(s)$ 定义为

$$\begin{cases} \text{Utility}(s) & \text{if Terminal-test}(s) \\ \max_{a \in \text{Actions}(s)} \text{minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{Max} \\ \min_{a \in \text{Actions}(s)} \text{minimax}(\text{Result}(s,a)) & \text{if Player}(s) = \text{Min} \end{cases}$$

- 如果状态是终结节点（terminal node）：该节点的值是 $\text{utility}(\text{state})$ （效用值）
- 如果状态是 MAX 节点：该状态的值是所有后继节点值（子节点）中的最大值
- 如果状态是 MIN 节点：该状态的值是所有后继节点值（子节点）中的最小值

The minimax algorithm

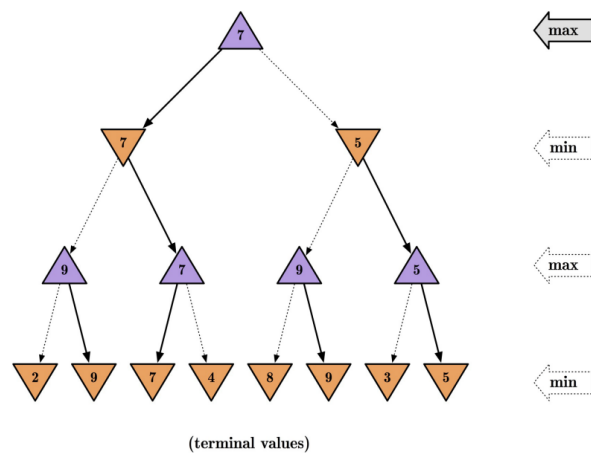
```

/* Find the child state with the lowest utility value */
function MINIMIZE(state)
  returns TUPLE of (STATE, UTILITY) :
    if TERMINAL-TEST(state):
      return (NULL, EVAL(state))
    (minChild, minUtility) = (NULL, ∞)
    for child in state.children():
      (_, utility) = MAXIMIZE(child)
      if utility < minUtility:
        (minChild, minUtility) = (child, utility)
    return (minChild, minUtility)

/* Find the child state with the highest utility value */
function MAXIMIZE(state)
  returns TUPLE of (STATE, UTILITY) :
    if TERMINAL-TEST(state):
      return (NULL, EVAL(state))
    (maxChild, maxUtility) = (NULL, -∞)
    for child in state.children():
      (_, utility) = MINIMIZE(child)
      if utility > maxUtility:
        (maxChild, maxUtility) = (child, utility)
    return (maxChild, maxUtility)

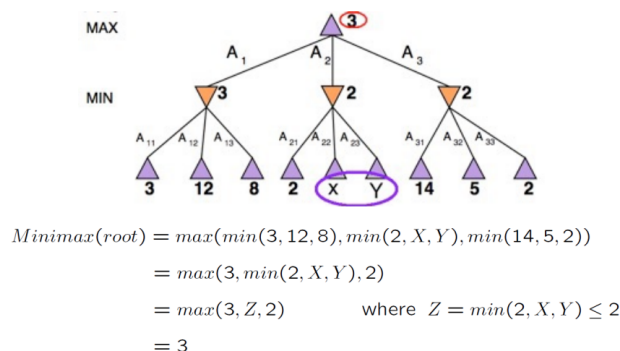
/* Find the child state with the highest utility value */
function DECISION(state)
  returns STATE :
    (child, _) = MAXIMIZE(state)
    return child
  
```

Minimax example



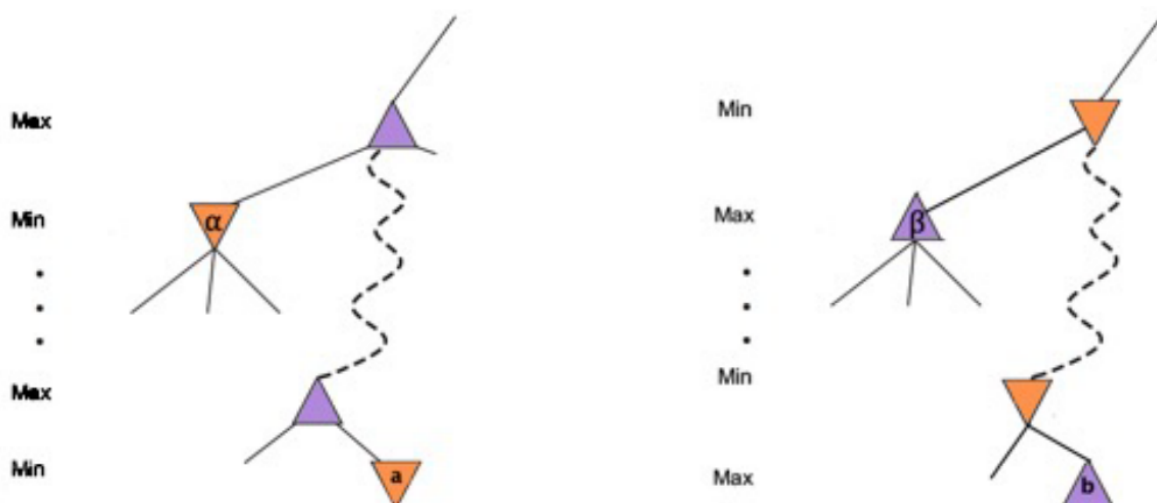
- 最优（假设对手也最优）且完备（有限树）
- 深度优先搜索时间： $O(b^m)$
- 深度优先空间： $O(bm)$
 - 井字棋（Tic-Tac-Toe）
 - 平均约 5 个合法走法，总共 9 步（9 个 ply）。
 - $5^9 = 1,953,125$
 - $9! = 362,880$ 个终结节点（注：视具体计数方式）
 - 国际象棋（Chess）
 - 平均分支因子 $b \approx 35$
 - 游戏树深度 $d \approx 100$ （典型游戏）
 - $b^d \approx 35^{100} \approx 10^{154}$ 节点
 - 围棋（Go）在 19×19 棋盘上起始分支因子约 361
- 问题：在真实游戏中，我们受时间限制，因此无法搜索到叶子节点
 - 为了实用并在合理时间内运行，minimax 只能搜索到某个深度。
 - 多一层（ply）会带来很大差异
 - 解决方案：
 - 用评估函数（evaluation function）替代非终结态的终结效用。
 - 使用迭代加深搜索（IDS）
 - 使用剪枝（pruning）：消除树上的大量分支
- α - β 剪枝（ α - β pruning）

$\alpha - \beta$ pruning



Minimax decisions are independent of the values of X and Y .²⁹

- 策略 (Strategy) : 和 minimax 一样, 以深度优先搜索进行。
- 参数 (Parameters) : 跟踪两个界限
 - α : 在当前从根到某条路径上, MAX 已经找到的最好 (即最大) 选择的值 (下界)。
 - β : 在当前从根到某条路径上, MIN 已经找到的最好 (即最小) 选择的值 (上界)。
- 初始化 (Initialization) : $\alpha = -\infty$, $\beta = +\infty$
- 传播 (Propagation) : 在搜索过程中把 α 、 β 的值沿路径向下传递以便用于剪枝。
 - 通过把终结节点值向上回传来更新 α 、 β 。
 - 仅在 Max 节点更新 α , 只有在 Min 节点更新 β 。
- 剪枝 (Pruning) : 每当 $\alpha \geq \beta$ 时即可剪掉剩余分支 (不再展开)。
-



- 若 α 对 MAX 来说比某分支 a 更好 (α 优于 a) , 那么 MAX 会避开该分支——也就是可以剪掉 (prune) 该分支。
- 若 β 对 MIN 来说比某分支 b 更好 (β 优于 b) , 那么 MIN 会避开该分支——也就是可以剪掉该分支。
- 直观: 当上层玩家已有比当前分支更好的保证时, 下层继续探索该分支没有意义, 可以省略。
-

$\alpha - \beta$ pruning

α, β 剪枝是在执行 minimax 的递归深度优先搜索过程中同时维护两个界 (α, β)，用它们来剪掉那些不可能改变最终节点选择分支子树，从而减少节点数量而不改变 minimax 结果。

```

/* Find the child state with the lowest utility value */
function MINIMIZE(state,  $\alpha$ ,  $\beta$ )
  returns TUPLE of (STATE, UTILITY) :
    if TERMINAL-TEST(state):
      return (NULL, EVAL(state))
    (minChild, minUtility) = (NULL,  $\infty$ )
    for child in state.children():
      (childUtility) = MAXIMIZE(child,  $\alpha$ ,  $\beta$ )
      if utility < minUtility:
        (minChild, minUtility) = (child, utility)
      if minUtility  $\leq \alpha$ :
        break
      if minUtility  $\geq \beta$ :
        break
       $\beta$  = minUtility
    return (minChild, minUtility)

/* Find the child state with the highest utility value */
function MAXIMIZE(state,  $\alpha$ ,  $\beta$ )
  returns TUPLE of (STATE, UTILITY) :
    if TERMINAL-TEST(state):
      return (NULL, EVAL(state))
    (maxChild, maxUtility) = (NULL,  $-\infty$ )
    for child in state.children():
      (childUtility) = MINIMIZE(child,  $\alpha$ ,  $\beta$ )
      if utility > maxUtility:
        (maxChild, maxUtility) = (child, utility)
      if maxUtility  $\geq \beta$ :
        break
      if maxUtility  $\leq \alpha$ :
        break
       $\alpha$  = maxUtility
    return (maxChild, maxUtility)

```

剪枝判断 (关键):

- if $minUtility \leq \alpha$ break (含义: 当前 MIN 节点已搜索到的最小值 α 来自上层 (对 MAX) 的 α 下界。因此 α 是 MAX 在其兄弟分支上能得到的值。若 MIN 的当前最小 α ，则 MAX 上层不会选择这个 MIN 节点的分支 (MAX 会选择至少 α 的分支)。因此继续遍历该 MIN 的其他子节点没有意义，可以中止循环，剪枝。)
- 更紧的上界值
- if $minUtility < \beta < maxUtility$ (假设 MIN 节点的当前最小值继续上升，更新传递到更上层 β)

/* Find the child state with the highest utility value */

function DECISION(state)

returns STATE : 初始调用，求节点

(child, _) = MAXIMIZE(state, $-\infty$, ∞)

return child

39

Move ordering (走子顺序)

- 影响 $\alpha - \beta$ 剪枝的有效性
- 例如：如果我们首先生成的是 D 的最差后继（对 Min 来说最差的孩子），则无法剪掉 D 的任何后继；但如果我们优先生成那个值为 2 的叶子（即最小的孩子）并先发现它，我们就能剪掉 D 的另外两个孩子（14 和 5）。
- 走子顺序的思想：优先检查那些最可能是最好的后继（即先探查那些可能导致更好 α / β 的子节点）。
- 最差的走子顺序 (Worst ordering)：不会发生任何剪枝（因为最好走法都在游戏树的右侧被发现）。复杂度仍为 $O(b^m)$ 。
- 理想的走子顺序 (Ideal ordering)：会发生大量剪枝（最好走法都排在树的左侧），实际复杂度可降到 $O(b^{\lceil m/2 \rceil})$ （实践中接近此值）。
- 如何找到好的走子顺序？
 - 记住浅层（较浅深度）搜索得到的最好走法。
 - 利用 domain knowledge 来排序走法（例如优先吃子、优先中心控制等）。
 - 保存/记录已经见过的状态（bookkeep the states），因为状态可能重复（可用置换表帮助按优先级重用旧信息）。

Real-time decisions

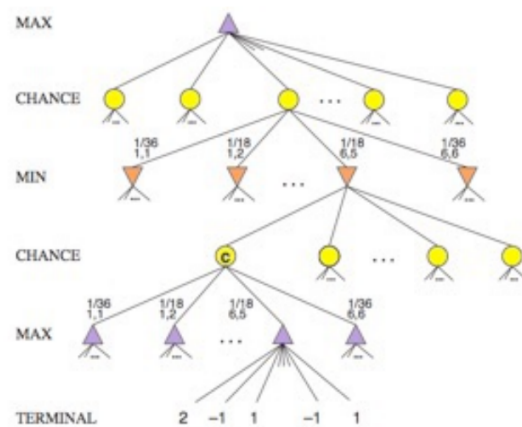
- minimax：需要生成完整的游戏搜索空间（理想情况）。
- $\alpha - \beta$ 算法：可以剪掉树上的大量分支（prune large chunks）。
 - 但 $\alpha - \beta$ 仍然需要一直搜索到叶子节点（到底层）。
- 在实时（实战）情形下不切实际（必须在有限时间内完成走子决定）。
- 解决方案：限制搜索深度（截断搜索），并用评估函数 eval(s) 来替代 terminal utility(s)，即用评估函数估计当前局面的价值。
- eval(s) 是在状态 s 上的启发式评估函数，用来把非终结节点当作“终结叶”估值。
 - Othello（翻转棋）：eval(s) = 白子数 - 黑子数
 - 国际象棋：eval(s) = 己方棋子总价值 - 对方棋子总价值（不同棋子有不同值）
- 理想的评估函数应当以与真实效用函数相同的方式对终结态排序，但又必须能快速计算。
 - 通常把评估函数表示为若干“特征”的线性加权和（或更复杂的学习模型）。

- 使用领域知识来设计最优且有用的特征
- 示例：
 - 选择有用的特征 $f_1, \dots, f_n$ ，例如在国际象棋中：棋盘上的棋子数、棋子价值（兵=1，主教=3 等）
 - 加权线性函数：

$$eval(s) = \sum_{i=1}^n w_i f_i(s)$$

- 从示例中学习权重 w_i
- Stochastic games（随机博弈 / 随机游戏）
 - 包含随机元素（例如掷骰子）
 - 包含“机会节点”（chance nodes）
 - Backgammon（西洋双陆棋）：一种将技巧与运气结合的古老棋盘游戏
 - 目标是每个玩家尽量将自己所有的棋子在对手之前从棋盘上移走

Stochastic games

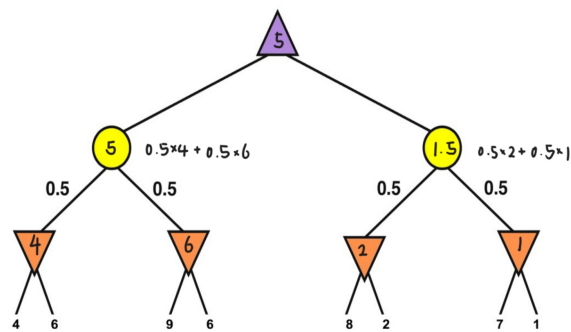


Partial game tree for Backgammon.

- 算法 Expectiminimax 把 Minimax 扩展以处理机会节点（chance nodes），具体如下：
 - 如果状态是 Max 节点，则返回其后继状态中最高的 Expectiminimax 值
 - 如果状态是 Min 节点，则返回其后继状态中最低的 Expectiminimax 值
 - 如果状态是机会节点，则返回其后继状态的 Expectiminimax 值的平均值（按概率加权）

Stochastic games

Example with coin-flipping:



Expectiminimax

For a state s :

Expectiminimax(s) =

$$\begin{cases} Utility(s) & \text{if Terminal-test}(s) \\ \max_{a \in Actions(s)} Expectiminimax(Result(s,a)) & \text{if Player}(s) = \text{Max} \\ \min_{a \in Actions(s)} Expectiminimax(Result(s,a)) & \text{if Player}(s) = \text{Min} \\ \sum_r P(r) Expectiminimax(Result(s,r)) & \text{if Player}(s) = \text{Chance} \end{cases}$$

- 这里 r 表示所有的概率事件（例如掷骰子的不同结果）， $Result(s, r)$ 表示在 s 状态下发生概率事件 r 后得到的新状态。

博弈：结论

- 在人工智能中，博弈被建模为一个搜索问题。
- Minimax 算法在假设对手也最优的情况下选择最佳着法。
 - Minimax 要把搜索走到底（遍历完整游戏树），在有时间限制的实战中不可行。
- Alpha-Beta 剪枝可以减少博弈树搜索量，从而在给定时间内允许更深的搜索。
 - 剪枝、记录表（bookkeeping）、评估启发式（evaluation heuristics）、节点重排序（node re-ordering）和迭代加深搜索（IDS）在实践中都很有效。
- 设计对抗性搜索智能体具有挑战性，因为状态空间非常庞大。
- 进一步可以探索的主题包括部分可观测博弈（如扑克牌类的牌类游戏，例如桥牌、扑克等）。